

Final Exam: PART 2

Simon Jones
COMS W3134
UNI: 12598
8/15/2021
Page 1 of 5

① Induction Proof

DEFINITION: Let a "complete undirected graph" be
 (what we already HAVE or know) a graph where every vertex is connected to every other vertex!

Ex



where $G = (V, E)$

WANT: To show that a complete undirected graph of N vertices will always have:

$$\left(\frac{N(N-1)}{2} \right) = \# \text{ of total edges in its graph (aka, } |E| \text{)}$$

BASE CASE: If $N=1$, then there exists ZERO total edges in this graph:

$$\left(\frac{1(1-1)}{2} \right) = \frac{0}{2} = 0 \text{ total edges in our "G"}$$

INDUCTIVE HYPOTHESIS: Assume that a "complete undirected graph", with $|V| = K$, has

$$|E| = \left(\frac{K(K-1)}{2} \right) \text{ total edges}$$

INDUCTIVE STEP: Now, consider the " $K+1$ "-numbered vertex. It must connect to our original " K -numbered" total vertices in our graph. So, an additional " K -numbered" edges are now required!

①	$\left(\frac{K(K-1)}{2} \right) + K$	(LHS)	{	$\left(\frac{(K+1)((K+1)-1)}{2} \right)$	(RHS)
②	$\frac{K(K-1)}{2} + 2K$		=	$\frac{(K+1) \times K}{2}$	
③	$\frac{K^2 - K + 2K}{2}$		=	$\frac{(K+1) \times K}{2}$	

NEXT

INDUCTIVE STEP
COMPS

LHS

RHS

(4)

$$\frac{K(K+1)}{2}$$

$$= \frac{K(K+1)}{2}$$

Srinan Jm
COMS W31BL
UNI: djs98
8/15/2021 5
Page 2 of 5

$$LHS = RHS \Rightarrow$$

Inductive Step
is proven

We have shown that

a "complete undirected graph" containing "N" vertices will always have " $\frac{N(N-1)}{2}$ "

total # of edges in its connected graph. QED

(2a) int C merge (int[] a, int[] b)

```
int sizeOfA = a.length;  
int sizeOfB = b.length;  
int x = 0;  
int y = 0;  
int z = 0;
```

```
int[] joinedArray = new int[sizeOfA + sizeOfB];
```

```
while (x < sizeOfA && y < sizeOfB)
```

```
{  
    if (a[x] < b[y])
```

```
    {  
        joinedArray[z++] = a[x++];
```

```
    }  
    else
```

```
    {  
        joinedArray[z++] = b[y++];
```

```
    }  
}
```

```
while (x < sizeOfA)
```

```
{  
    joinedArray[z++] = a[x++];
```

```
}
```



```
while (y < sizeOfB)
```

```
{  
    joinedArray[z++] = b[y++];  
}
```

```
return joinedArray;
```

```
}
```

Jinka Sare
COMS W3134
UNI: dj2509
8/15/2021
Page 3 of 5

```
(2b) int[] merge3 (int[] a, int[] b, int[] c)
```

```
{  
    int[] joinedArray1 = merge(a, b);  
    int[] joinedArray2 = merge(joinedArray1, c);
```

```
    return joinedArray2;  
}
```

```
(3) TreeNode minLeaf (TreeNode root)
```

```
{  
    if (root == null)
```

```
    {  
        return null;
```

```
    }
```

```
    else
```

```
    {  
        if (firstChild == null && nextSibling == null)
```

```
        {  
            return root;
```

```
        }
```

```
        else
```

```
        {  
            TreeNode leftMin = minLeaf (root.firstChild);
```

```
            TreeNode rightMin = minLeaf (root.nextSibling);
```

```
            if (leftMin != null && (rightMin == null || leftMin.data < rightMin.data))
```

```
            {  
                return leftMin;
```

```
            }
```

```
            else
```

```
            {  
                return rightMin;
```

NEXT →

< minLeaf() cont'd >

}

}

// closes minLeaf() method

④ boolean checkMinHeap (int A[])

{
 int nIntegers = A.length - 1;
 for (int i = 1; i <= nIntegers; i++)

{
 if (2*i <= nIntegers)

{
 if (A[i] > A[2*i])

{
 return false;

}

if (2*i + 1 <= nIntegers)

{
 if (A[i] > A[2*i + 1])

{
 return false;

}

}
 return true;

// closes checkMinHeap() method

⑤ ① If the TSP is the version where we want to return the shortest path cycle (source: ~~x~~ destination: A) then we are talking about the (NP-complete) version of the TSP algorithm.

② If our TSP "cannot be solved any faster than in $O(N^3 \times 2^N)$ time, then its Best Case Big-O time complexity becomes $O(N^3 \times 2^N) \sim O(N^3) \times O(2^N)$

Given:
P $\supset$ NP
NP $\supset$ P

Known:
If P $\supset$ NP & NP $\supset$ P are true, then:
P = NP

$\sim O(2^N)$ (exponential time complexity!!!)
this term dominates for large "N"

3) Thus, ^{if} TSP cannot be solved in polynomial time,

no NP-complete problem can be proven to exist with polynomial time complexity.

5 min
cons W35
WMI: d. 23/10
8/15/2022
Page 5 of 5

4) Because all NP-complete problems have not been shown, NP $\supset$ P is true.

5) Therefore, $P \neq NP$. no solution algorithm (w/ polynomial time complexity) exists for ANY NP-complete problem.

QED